GOV•UK

Guidance

API technical and data standards

Design, build and operate APIs in a consistent way

From: **Government Digital Service (/government/organisations/government-digital-service)** and **Central Digital and Data Office (/government/organisations/central-digital-and-data-office)**

Published 7 February 2018

Last updated 19 July 2024 —

Contents

— Design your API

— Build your API

— Operate your API

These standards are for people building Application Programming Interfaces (https://www.gov.uk/service-manual/technology/application-programming-interfaces-apis) (APIs) in government who want to:

- save time and resources
- assure users that their service meets minimum standards
- use agile methods (https://www.gov.uk/service-manual/agile-delivery) to improve products and services
- use the REST API style
- take an API-first approach to development

You should use these standards when designing, building and operating your APIs for use in government and public services. This will ensure that your APIs work better across different platforms and services.

Design your API

Gather user needs

Before you build an API, you must understand the needs of your users.

For an API, the user is a developer who wants to consume your API to deliver a service. The developer will have needs based on:

- the service they are developing
- how easy it is to write code to consume your API

Making your API easy to understand means it's more likely to be used, because developers may not read all of your documentation.

Starting with user needs (https://www.gov.uk/service-manual/user-research/start-by-learning-user-needs) will allow you to focus on simplifying the interface, removing any features that are not useful to users.

Check for existing APIs

It's faster and simpler to reuse an existing API than build one from scratch. You should only build a new API when necessary.

You can check if there are existing internal, external or commercially available APIs by looking at internal API catalogues and the cross-government UK API Catalogue (https://www.api.gov.uk/#uk-public-sector-apis).

Make sure that any API you choose will have the right functionality to work for your use case. You must also check the licence it is provided under, to make sure you are able to use it in your service. For example, an open API provided under an open licence (https://spdx.org/licenses/) can be used with few restrictions, but commercial APIs may have usage limits.

Read more about using open licences for government services (https://www.gov.uk/service-manual/technology/making-source-code-open-and-reusable#licensing-your-code).

Design your API first

API first is the practice of designing software starting with an API, before designing your web or mobile user interface.

Developing the API before the rest of the service means a platform or service can be built around the API. This will reduce the need to repeat work, if later on an API is required for your service.

APIs are often an afterthought – built when a service already exists, as a way for other services to access its data. If you're building an API to a legacy system, that may be your only choice, but you should still think about the user needs for your API.

Developing your API before any other interfaces has other advantages, including:

- other services being able to use your API
- stress-testing of your API by your own internal services, allowing you to make continuous improvements – this will improve developer experience by exposing the complexities of the API and making sure, for example, that documentation is fit for purpose
- improved modularity and reuse of code, as the API will not have to be customised to fit an existing service – this leads to more consistent interfaces, meaning developers can be more comfortable with your API and can speed up integration
- the resources your API exposes will be fit for purpose – by starting with the API, it means that the business logic of your service can be clearly separated from the data structures used in any underlying data store

Follow the Technology Code of Practice and other standards

You should follow the Technology Code of Practice (https://www.gov.uk/guidance/the-technology-code-of-practice) when designing any technology in government, including APIs. Point 10 – Make better use of data (https://www.gov.uk/guidance/make-better-use-of-data) may be particularly relevant when you're working with APIs.

You also need to design your API to follow all appropriate government data standards in the Data Standards Catalogue (https://alphagov.github.io/data-standards-authority/standards/).

The following standards are especially useful:

- ISO 8601 standard (https://www.gov.uk/government/publications/open-standards-for-government/date-times-and-time-stamps-standard) – this represents the date and time in your API's responses, preventing issues with ambiguous date formats
- GeoJSON (http://geojson.org/) – use this format for encoding and exchanging location information

The UK Geospatial Data Standards Register (https://www.gov.uk/government/publications/uk-geospatial-data-standards-register/national-geospatial-data-standards-register#coordinate-reference-systems) has details of coordinate reference systems that you should use when exchanging location information through your API.

You should also follow the guidance related to:

- UK General Data Protection Regulation (GDPR) (https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/)
- privacy and data protection by design (https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/accountability-and-governance/guide-to-accountability-and-governance/accountability-and-governance/data-protection-by-design-and-default/)

Use the REST API style

Representational state transfer (REST) means an API follows the REST architectural style (https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm), and works with REST (sometimes called RESTful) web services. This can make the API easier to use and faster to integrate with.

REST principles are widely adopted, which means developers will understand your API more quickly. It also means you can use industry-standard ways of documenting your API, and off-the-shelf testing software.

When you're using the REST API style, you should follow the REST design principles:

- uniform interface – all API requests for the same resource should have the same uniform resource identifier (URI)
- client and server – these must be independent of each other
- statelessness – this means that all requests and responses are self-contained and include all necessary information, and that no server-side sessions are required because all session state is kept on the client side

- cacheable – resources should be cached on the client side and server side, to improve performance and scalability
- layered system – allows for components such as proxies, gateways and firewalls to be placed between the client and the server, to make the service more reliable and secure

Using REST is a good way to design an API, but other approaches such as GraphQL (https://www.gov.uk/guidance/using-graphql-for-your-api) or gRPC (https://grpc.io/) may still be a better choice for specific projects. For example, GraphQL is useful for prototyping services, when you're not sure what views of the data other developers will need.

Develop a specification before you start to code

The OpenAPI Specification (https://swagger.io/specification/) is a standardised way of describing RESTful web APIs, and is recommended (https://www.gov.uk/government/publications/recommended-open-standards-for-government/describing-restful-apis-with-openapi-3) by the government Open Standards Board.

It allows you to produce a file (the OpenAPI document) which is both machine and human readable, and which describes the format and responses of your API.

A good principle is to produce an OpenAPI document as the first output of your design process, and then develop the document along with the API design.

This will help to:

- show the API has been developed consistently with others from your organisation
- test the API against rules and for security issues using a software linter (https://owasp.org/www-project-devsecops-guideline/latest/01b-Linting-Code#:~:text=Linting%20is%20the%20automated%20checking,(otherwise%20known%20as%20linter).)
- generate reference documentation automatically – documentation generated in this way should be supported by further resources (https://www.gov.uk/guidance/how-to-document-apis)

For non-REST APIs like GraphQL, you should still look to produce a specification during the design process. For example, for GraphQL this would

be a GraphQL schema specification defining the types available, queries, mutations and the relationships between them.

Be secure by design

You must think about security from the very beginning of the design process for your API, and follow a secure by design approach (https://www.security.gov.uk/guidance/secure-by-design/).

API security involves:

- data level security – making sure users only have access to the data provided by the API that they are authorised to see
- application level security – making sure only authorised users can access the API
- network security – making sure only trusted clients can consume your API
- auditing – making sure the usage of the API is monitored

The OWASP foundation has a list of top 10 security risks for APIs (https://owasp.org/www-project-api-security/). Make sure you avoid these when designing your API.

Test your assumptions with users

By creating your OpenAPI specification first, before building your API, you can use tools that automatically generate test stubs of your API.

Test stubs allow you to:

- test your design choices with the potential consumers of your API
- build out a suite of functional tests for your API

This can shorten the feedback loop and allow you to refine your API in the design stage without spending time and effort building out the real API. It will also help you follow a test-driven development (TDD) approach to building your real API.

There are many different tools that can help when designing and testing your API. Some popular tools are:

- SwaggerHub
- Swagger.io

- Postman
- Bruno
- ReadyAPI

All have a free tier, but charge for more enterprise features.

There are also many open source tools that are free to use – for example, the openapi.tools GitHub repository (https://github.com/apisyouwonthate/openapi.tools) maintains lists of open source API tooling.

When using any cloud-based API tool, be sure to check how the tool will keep your API credentials secure before sharing them.

Iterate the design based on feedback

Follow an agile process (https://www.gov.uk/service-manual/agile-delivery) when designing your API. You do this by incrementally building out your design and continuously testing it and getting feedback from the people who will be consuming it.

At the design stage it's easy to make changes to your API. You can entirely remove endpoints and redefine their response formats without worrying about breaking changes.

Once your API is built and people start using it, it becomes much harder to make big changes because you need to consider backward compatibility between versions of your API.

Build your API

Use the UTF-8 standard to encode your API

Unicode (https://home.unicode.org/) is the world standard for consistently encoding, representing and handling text in most global writing systems.

You should use the Unicode Transformation Format (UTF-8) standard (https://www.gov.uk/government/publications/open-standards-for-government/cross-platform-character-encoding-profile) when encoding unicode character sets. This will help you read, write, store and exchange text that will remain stable over time and across different technologies.

Use JSON for response formats

Where possible, you should use the JSON Data Interchange Standard (https://datatracker.ietf.org/doc/html/rfc8259) when structuring your REST API's response formats.

There are several JSON formats in use today. If your organisation has not already specified one, we recommend JSON:API (https://jsonapi.org/). This is specifically designed for API responses, and has the benefits of making some design choices for you by specifying conventions.

If you are working with a legacy API that does not provide JSON responses – for example, a SOAP API that uses XML, it may be better for your users if you keep the same format. You should make sure this is well documented because XML is increasingly uncommon.

Choosing a broadly-adopted standard whenever possible gives you the advantages of:

- saving time by avoiding debate about what format to use
- allowing you to follow industry best practices
- making it easier for external developers to integrate with your API

Use consistent names for resources

Developers should be able to assume the names of the resources in your API from context, so name each type of resource consistently. For example, if a resource represents a collection, choose whether this should always be singular or plural (for example: order or orders).

You should also use naming conventions for similar resources. For example, if you have a user and address resource, the name you use for the id of each should match: user_id and address_id.

Your users should not have to reread documentation to be able to know what the name of a particular resource is.

Make your resources persistent

The names of the resources your API provides should not change between versions, as this could break integrations.

Make sure your API provides a level of abstraction from the underlying data sources. It should not matter if the columns in your database change name, because the API should provide a map from the resource name to the underlying data.

Use standard HTTP responses

Make sure you match error codes with standard HTTP response codes (https://datatracker.ietf.org/doc/html/rfc9110#name-status-codes).

Your error codes must be consistent and easy to read, so that it's clear where an error has occurred. There are often cases where the same API endpoint could return the same HTTP status code for different conditions, so descriptive error messages will help users understand what's gone wrong.

You should document all error codes and make sure they're easy to find.

Any custom error codes should only contain information needed to diagnose the problem. Do not include non-essential information which could help an attacker target the service – for example, technical details of the system the API is running on.

Host your API

When you build an API, it's important to think about where you will host it and how it will continue to work during its lifecycle.

When you name and host your API, as well as its documentation, you should follow guidance on choosing an API domain name (https://www.gov.uk/guidance/get-an-api-domain-on-govuk).

Control access to your API

When you build your API, you need to decide the best way to give users access. In general you should make all users of your API authenticate their identities. This is essential if your API deals with personal or sensitive data.

Avoid endpoints that allow anonymous users, as these can:

- increase the attack surface available to hackers (https://www.security.gov.uk/guidance/secure-by-design/principles/#7-minimise-the-attack-surface)
- make it harder to monitor users that are consuming excessive resources

If you require anonymous endpoints, their responses should be limited to open data. For example the GOV.UK Content API (https://content-api.publishing.service.gov.uk/) does not require authentication, but only returns metadata and the HTML content of pages published on GOV.UK. Anonymous endpoints should also be rate limited to prevent excessive or malicious use.

Use the industry standard OAuth 2.0 Authorization Framework (https://datatracker.ietf.org/doc/html/rfc6749) to manage access to your API. This will make it easier for users to consume your API while giving you better control over the level of access they have.

Never use basic authentication because usernames and passwords are sent encoded, but unencrypted, in the HTTP header. This makes it easy for an attacker to steal them.

You should also avoid using API keys. An API key is a unique identifier that is issued to API users. It needs to be sent with every request – either in the URL, as a request header or as a cookie – and it can easily be intercepted by an attacker and reused. If you do use API keys you should time limit their use and regularly change the keys to keep them secure.

Neither basic nor API key authentication is secure unless used in conjunction with HTTPS.

OAuth 2.0 typically uses digitally signed security tokens in JWT (https://datatracker.ietf.org/doc/html/rfc7519) (JSON Web Token) format that are passed in the Authenticate header of a request, making them harder to tamper with.

OAuth 2.0 defines ways to authenticate different types of API clients:

- use client credentials (https://datatracker.ietf.org/doc/html/rfc6749#section-1.3.4) when another service or application is consuming your API, outside of the context of a user
- use authorization code (https://datatracker.ietf.org/doc/html/rfc6749#section-1.3.1), with the PKCE (https://datatracker.ietf.org/doc/html/rfc7636) extension, when a user accesses your API – for example, through a web application

As well as authenticating your users, you should define resource-level access controls for your API, and check the authorisation for every request.

When using OAuth 2.0 this means:

- a user must first request the scope of access they need to your API – for example, an order-read or order-write scope

- on each call to your API you should check that the user has the required scope before granting them access

Defining scopes (https://datatracker.ietf.org/doc/html/rfc6749#section-3.3) in this way allows for fine-grained access control to the endpoints of your API, and means you can include the authorisation information in your OAS specification (https://spec.openapis.org/oas/latest.html#oauth-flow-object). This makes it easier for consumers to use your API securely.

Authentication and authorisation are features that can be defined and controlled through an API management system. Find out more about how to manage operations with an API gateway (https://www.gov.uk/guidance/defining-an-api-management-strategy#managing-operations-with-an-api-gateway).

Secure your API

Follow the GOV.UK Service Manual guidance on using HTTPS (https://www.gov.uk/service-manual/technology/using-https) when serving your API over the web, to make it as secure as possible.

You must use TLS 1.2 or above to secure your API. The National Cyber Security Centre has guidance on using TLS to securely deliver web services (https://www.ncsc.gov.uk/guidance/using-tls-to-protect-data).

Validate all inputs (https://www.ncsc.gov.uk/collection/cyber-security-design-principles/making-compromise-difficult) to your API. Endpoints that de-serialise data should enforce schema validation and reject unknown attributes. All parameters (URL and query string) should be validated and type checked before being processed. Otherwise attackers could exploit lax input validation to craft requests that result in unexpected effects.

Configure your API using appropriate Cross-Origin Resource Sharing (CORS) headers (https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS). This will minimise the risk of cross-origin attacks.

Turn off unnecessary HTTP verbs (GET, PUT, POST, etc), and only allow the verbs your API actually supports on each resource. For example, if users can never be deleted, do not accept DELETE on the users resource.

Remove any endpoints you do not need – for example, test endpoints or stubs for future development – and only expose the endpoints users will actually use.

You can also enforce your content type. For example, if your API only accepts JSON then enforce this through content type headers and reject other types

of request.

For a complete list of common security issues to fix in your API, refer to the OWASP API Security Project (https://owasp.org/www-project-api-security/).

Consider performance and scalability

You can measure your API's performance by how fast it can deal with a single request and make a response. Its scalability is the amount of requests it can deal with at the same time while maintaining an acceptable performance level.

You can improve the performance and scalability of your API response by making it cacheable. This means the API response can store copies of frequently-accessed data along its request and response path. This reduces bandwidth, latency and server load, as well as making network failures less of a problem for your users.

A cacheable API response has other advantages, for example letting you use a Content Delivery Network (CDN). A CDN can make your API faster and more reliable by caching data for responses in locations that are closer to the user.

You should also implement rate limiting or throttling policies – making sure that users don't overuse your resources

Provide an API test service

You should try to offer a testing service (also known as a sandbox) for your users. They may find an API harder to integrate without a test service, so providing one is a good use of your project's time and budget.

Sandboxes can also be generated much more easily with an OpenAPI document. There are tools available that will convert these documents into test endpoints.

API test services should still be secure and require all users to be authenticated before granting access. Many API management solutions include developer portals that allow developers to create development accounts which they can use to access test services.

Test services should never contain real personal or transactional data, but could use real reference data – for example, lists of local authorities.

Having a test service available also means your developers can:

- start to get comfortable with an API in a sandbox environment very early in its development
- work while other parts of the project are being completed – for example, they can work with test data in the test service while a data sharing agreement is still being drawn up to access the real data for the finished API

However, providing a sandbox of good quality can be complicated so consider your options carefully if you're going to do it. For example, if you plan to create synthetic data, you may need to use a paid service.

A good sandbox should include:

- useful test data that reflects the real API
- implementation and dependencies behaviour that is properly simulated
- the ability to save, protect and restore data

Test your API's compliance

You must make sure your API meets the legal requirements of your organisation.

After you've tested your API's compliance, you should make it easy to find (https://www.gov.uk/guidance/defining-an-api-management-strategy#making-your-apis-easy-to-find) by adding it to your organisation's API catalogue and the cross-government API Catalogue (https://www.api.gov.uk/#uk-public-sector-apis), so that others can use it.

Document your APIs

Your team's developer, user researcher and technical writer should work together on your API's documentation.

Follow the guidance on:

- documenting APIs (https://www.gov.uk/guidance/how-to-document-apis)
- writing API reference documentation (https://www.gov.uk/guidance/writing-api-reference-documentation)

At the end of the build phase, it's always useful to carry out a quick evaluation. Some of the questions you might ask include:

- was UTF-8 used for all text encoding?
- how were dates and times represented?
- how is user-level authorisation managed?
- what are the limits of scalability of your API?
- is the documentation easy to understand?

Operate your API

Version your API and support older versions

When you make new versions of your API, try not to make changes that will stop older versions of your API working properly. If you cannot keep older versions working you should expose a new version of your API by adding the version number into the URI, for example https://myapi.service.gov.uk/v1 (https://myapi.service.gov.uk/v1).

URI versioning is the simplest and most commonly used way to version an API.

Other ways to version an API include using a custom header or defining a custom media type. Avoid these approaches because they can lead to your API being blocked by proxies or firewalls.

You should try to keep the number of active versions of your API to a minimum, and encourage users to move to the latest version, to reduce the overhead of maintaining multiple versions.

You can think about retiring older versions of an API (https://www.gov.uk/guidance/defining-an-api-management-strategy#taking-your-api-out-of-service) if you can tell from the logs that only a few people are using them anymore. When you retire an old version you must tell users it is no longer supported and give them time to move on to the new version.

Use an API management system or gateway

An API management platform provides services for your API that it rarely makes sense for you to build yourself. For example, access control and authorisation, audit and logging, and network management.

In production all of these services are important, and can be more reliably and easily provided by an API management tool or gateway

(https://www.gov.uk/guidance/defining-an-api-management-strategy#managing-operations-with-an-api-gateway) – of which there are several open source options.

Log your API's use

If your API provides personal or sensitive data, you must log when the data is provided and who you provide it to.

This will help you:

- follow UK GDPR (https://ico.org.uk/for-organisations/data-protection-and-the-eu/data-protection-and-the-eu-in-detail/the-uk-gdpr/)
- respond to data subject access requests
- detect fraud or misuse

An API management tool can help with logging and auditing usage of your API.

Monitor APIs for unusual activity

You must make sure all technology in your team or organisation is secure. APIs are no exception. The Technology Code of Practice explains how to make things secure (https://www.gov.uk/guidance/make-things-secure). Using the logging and audit tools from your API gateway will help you recognise when use of the API changes over time.

Published 7 February 2018
Last updated 19 July 2024 + show all updates

OGL

All content is available under the Open Government Licence v3.0, except where otherwise stated

© Crown copyright